

JAVA 8 MASTER HANDBOOK

Lecture 1 - Java 8 Introduction

Part 1 - Why Java 8 Was Needed & Evolution of Java

Goal of Part 1:

Is part ke end tak tum ye samajh jaoge:

- Java 8 ki zarurat kyun padi?
- Java 7 me kya problems thi?
- Java 8 ka main objective kya tha?
- Functional Programming kya hoti hai?
- Java ne apni philosophy kyun change ki?

Table of Contents

1. Evolution of Java
2. Why Java 8 Was Needed
3. Problems Before Java 8
4. Boilerplate Code
5. Code Duplication Problem
6. Software Engineering Principle
7. Java's Biggest Evolution
8. OOP vs Functional Programming
9. Java 8 Philosophy
10. Key Takeaways
11. Interview Questions
12. Revision Notes

Chapter 1 - Evolution of Java

Programming language kabhi bhi ek hi version me perfect nahi hoti.

Har naye version ka ek purpose hota hai.

Chalo Java ki journey dekhte hain.

Version	Major Feature
Java 1.0	OOP, JVM

Version	Major Feature
Java 5	Generics, Enhanced For Loop, Annotations
Java 7	Try-with-resources, Diamond Operator
Java 8	Lambda Expressions, Stream API, Functional Programming Support

Observe karo.

Java 8 sirf ek normal update nahi tha.

Ye Java ki history ka **sabse bada evolution** tha.

Isliye interviews me aksar pucha jata hai:

Which Java version changed Java programming style the most?

Answer

Java 8

Why Java 8 Is Called Revolutionary?

Java 8 ne sirf naye classes add nahi ki.

Usne programming ka pura style badal diya.

Java 7 tak

Object-Oriented Programming

Java 8 ke baad

Object-Oriented Programming

+

Functional Programming

Isliye Java 8 ko

The biggest release in Java history

bola jata hai.

Before Java 8

Java ki philosophy simple thi.

Everything revolves around

Class

↓

Object

↓

Method

↓

Execution

Example

```
class Student{  
  
}
```

Object

```
Student s = new Student();
```

Method

```
s.study();
```

Har cheez object ke through.

Isliye Java ko Object-Oriented language bola jata tha.

Real World Analogy

Suppose tum restaurant me gaye.

Old Java philosophy

Customer

↓

Manager

↓

Chef

↓

Food

Har step manually.

Ye Object-Oriented style hai.

Java Developers Faced a Problem

Software companies ne notice kiya.

Applications badi hoti ja rahi thi.

Code bhi bada hota ja raha tha.

Aur sabse dangerous cheez...

Duplicate code.

Ye samajhna bahut important hai.

Chapter 2 - Duplicate Code Problem

Suppose tumhare paas

1

2

3

4

5
6
7
8
9
10

Ab tumhe even numbers print karne hain.

```
for(int i=1;i<=10;i++){  
    if(i%2==0){  
        System.out.println(i);  
    }  
}
```

Simple.

Ab odd numbers.

```
for(int i=1;i<=10;i++){  
    if(i%2!=0){  
        System.out.println(i);  
    }  
}
```

Ab multiples of 3.

```
for(int i=1;i<=10;i++){  
    if(i%3==0){  
        System.out.println(i);  
    }  
}
```

```
}  
  
}
```

Observe carefully.

Kya change hua?

Sirf

Condition

Baaki

- Loop
- Printing
- Traversal

Same.

ASCII Diagram

```
Loop  
|  
├─ Condition A  
  
Loop  
|  
├─ Condition B  
  
Loop  
|  
├─ Condition C
```

Sirf condition badal rahi hai.

Baaki pura code same hai.

Software Engineering Perspective

Interview me ek term use hota hai.

Boilerplate Code

What is Boilerplate Code?

Boilerplate code wo code hota hai jo baar-baar repeat hota hai aur jisme logic bahut kam change hota hai.

Example

```
Loop  
  
Loop  
  
Loop  
  
Loop  
  
Loop
```

Har jagah loop.

Condition alag.

Ye boilerplate hai.

Real Project Example

Suppose Amazon me products hain.

```
Product A  
  
Product B  
  
Product C  
  
Product D
```

Ab requirement aayi.

Price > 1000

```
for(Product p : products){  
    if(p.getPrice()>1000){
```

```
    }  
}
```

Rating > 4

```
for(Product p : products){  
    if(p.getRating()>4){  
        }  
}
```

Category = Electronics

```
for(Product p : products){  
    if(p.getCategory().equals("Electronics")){  
        }  
}
```

Stock > 10

```
for(Product p : products){  
    if(p.getStock()>10){  
        }  
}
```

Observe.

Loop

Same.

Condition

Different.

Problem Visualization

Loop

↓

Condition

↓

Result

Again

Loop

↓

Condition

↓

Result

Again

Loop

↓

Condition

↓

Result

Java developers ne socha.

Hum har baar

- Loop
- Loop
- Loop

- Loop

kyun likh rahe hain?

Chapter 3 - Employee Example (Interview Favourite)

Suppose company me employees hain.

```
Rahul  
  
Salary = 40000  
  
Department = IT  
  
Age = 25
```

```
Amit  
  
Salary = 90000  
  
Department = HR  
  
Age = 40
```

```
Riya  
  
Salary = 60000  
  
Department = IT  
  
Age = 35
```

Requirement 1

Salary > 50000

```
for(Employee e : employees){  
  
    if(e.getSalary()>50000){  
  
        System.out.println(e);  
  
    }  
}
```

```
}  
  
}
```

Requirement 2

Department = IT

```
for(Employee e : employees){  
  
    if(e.getDepartment().equals("IT")){  
  
        System.out.println(e);  
  
    }  
  
}
```

Requirement 3

Age > 30

```
for(Employee e : employees){  
  
    if(e.getAge()>30){  
  
        System.out.println(e);  
  
    }  
  
}
```

Requirement 4

Salary > 70000

```
for(Employee e : employees){  
  
    if(e.getSalary()>70000){  
  
        System.out.println(e);  
  
    }  
  
}
```

```
}  
  
}
```

Question

Loop kitni baar likha?

Answer

4

Question

Condition kitni baar change hui?

Answer

4

Question

Loop change hua?

Answer

NO

Yahi Java 7 ki sabse badi problem thi.

Software Engineering Principle

Ek bahut famous principle hai.

Behavior should be passed, not duplicated.

Is line ko yaad kar lo.

Ye Java 8 ki foundation hai.

Iska matlab kya hai?

Loop ko baar-baar mat likho.

Sirf condition badlo.

Aur wahi condition pass kar do.

Pseudo Code

```
filter(employeeList, salaryCondition)
```

Kal

```
filter(employeeList, ageCondition)
```

Parso

```
filter(employeeList, departmentCondition)
```

Same method.

Sirf behavior alag.

Isi behavior ko Java 8 me **Lambda Expressions** ke through represent kiya gaya.

Chapter 4 - Java's Biggest Evolution

Java developers ne ek bahut bada decision liya.

Pehle Java me hum objects pass karte the.

Example

```
Collections.sort(list, comparatorObject);
```

Comparator bhi ek object hota tha.

Java 8 ke baad idea ye tha:

"Kya hum object ki jagah **behavior** pass kar sakte hain?"

Behavior ka matlab:

- Compare ka logic

- Filter ka logic
- Sort ka logic
- Validation ka logic

Ye hi soch aage chal kar Lambda Expressions ki foundation bani.

Important Interview Note

Interviewer:

Java 8 ka core idea kya tha?

Expected Answer

Java 8 ka core idea tha **behavior ko reusable banana**, boilerplate code ko kam karna, aur Object-Oriented Programming ke saath Functional Programming features introduce karna.

Common Misconception

✗ Java 8 sirf Stream API ke liye famous hai.

Ye incomplete answer hai.

✓ Better answer:

Java 8 ka foundation **Lambda Expressions** hain.

Streams, Method References, Collectors aur kai modern APIs Lambda Expressions aur Functional Interfaces ke concept ko use karte hain.

Quick Revision Box

Java 7 Problems

↓

Boilerplate Code

↓

Duplicate Loops

↓

Poor Readability

↓

Hard Maintenance

↓

Java 8 Introduced

↓

Behavior Passing

↓

Lambda

↓

Stream API

Interview Questions

Q1. Why was Java 8 needed?

Answer

Java 7 me boilerplate code bahut tha.

Collection processing ke liye baar-baar loops likhne padte the.

Java 8 ne Lambda Expressions aur Stream API ke through code ko concise, readable aur maintainable banaya.

Q2. What is Boilerplate Code?

Answer

Aisa repeated code jo har jagah same hota hai aur sirf chhota sa logic change hota hai.

Jaise baar-baar same `for` loop likhna aur sirf `if` condition badalna.

Q3. What was the biggest problem before Java 8?

Answer

- Repeated loops
 - Duplicate logic
 - Anonymous classes ka verbose syntax
 - Collection processing ka imperative style
-

Q4. What is meant by "Behavior should be passed, not duplicated"?

Answer

Logic ko baar-baar likhne ke bajay usse reusable behavior ke roop me pass karo.

Java 8 me ye kaam Lambda Expressions aur Functional Interfaces ki help se hota hai.

Part 1 Summary

Is Part me humne samjha:

- Java 8 Java ka sabse bada evolution tha.
- Java 7 me boilerplate aur duplicate code ki problem thi.
- Software engineering ka principle:

Behavior should be passed, not duplicated.

- Isi problem ko solve karne ke liye Java 8 ne Functional Programming concepts introduce kiye.
 - Lambda Expressions aur Stream API isi philosophy ka result hain.
-

Homework

1. Boilerplate code kya hota hai?
 2. Duplicate code aur boilerplate code me kya difference hai?
 3. Java 7 me collections process karna difficult kyun tha?
 4. "Behavior should be passed, not duplicated" ko apne words me explain karo.
 5. Employee example ko khud modify karke ek aur real-world example banao (Student, Product ya Order system).
-

Next Part

Lecture 1 – Part 2

Topics:

- Functional Programming (Deep Dive)
- Imperative Programming
- Declarative Programming
- Java 8 Philosophy
- Why Functional Programming Was Introduced
- Foundation of Lambda Expressions